Expert Systems With Applications 85 (2017) 123–133

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Mining top-*k* co-occurrence items with sequential pattern

Tung Kieu[a], Bay Vo[b,c], Tuong Le[d,e,*], Zhi-Hong Deng[f], Bac Le[a]

[a] Faculty of Information Technology, University of Science, VNU-HCM, Viet Nam
[b] Faculty of Information Technology, Ho Chi Minh City University of Technology, Viet Nam
[c] College of Electronics and Information Engineering, Sejong University, Seoul, Republic of Korea
[d] Division of Data Science, Ton Duc Thang University, Ho Chi Minh City, Viet Nam
[e] Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Viet Nam
[f] Key Laboratory of Machine Perception (Ministry of Education), School of Electronics Engineering and Computer Science, Peking University, Beijing 100871, China

ARTICLE INFO

Article history:
Received 25 January 2017
Revised 7 May 2017
Accepted 8 May 2017
Available online 10 May 2017

Keywords:
Top-*k* mining
Co-occurrence sequential mining
Sequential pattern mining

ABSTRACT

Frequent sequential pattern mining has become one of the most important tasks in data mining. It has many applications, such as sequential analysis, classification, and prediction. How to generate candidates and how to control the combinatorically explosive number of intermediate subsequences are the most difficult problems. Intelligent systems such as recommender systems, expert systems, and business intelligence systems use only a few patterns, namely those that satisfy a number of defined conditions. Challenges include the mining of top-*k* patterns, top-rank-*k* patterns, closed patterns, and maximal patterns. In many cases, end users need to find itemsets that occur with a sequential pattern. Therefore, this paper proposes approaches for mining top-*k* co-occurrence items usually found with a sequential pattern. The Naive Approach Mining (NAM) algorithm discovers top-*k* co-occurrence items by directly scanning the sequence database to determine the frequency of items. The Vertical Approach Mining (VAM) algorithm is based on vertical database scanning. The Vertical with Index Approach Mining (VIAM) algorithm is based on a vertical database with index scanning. VAM and VIAM use pruning strategies to reduce the search space, thus improving performance. VAM and VIAM are especially effective in mining the co-occurrence items of a long input pattern. The three algorithms were evaluated using real-world databases. The experimental results show that these algorithms perform well, especially VAM and VIAM.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

Agrawal and Srikant (1995) introduced the problem of mining frequent sequential patterns (FSPs). Given a sequential database, where each sequence consists of a list of itemsets and each itemset consists of a set of items, and a user-specified minimum support threshold (*minsup*), the goal of FSP mining is to find all sequences whose supports are greater than or equal to *minsup*. Various methods have been proposed for mining FSPs (Ayres, Flannick, Gehrke, & Yiu, 2002; Fournier-Viger, Lin, Kiran, Koh, & Thomas, 2017; Fournier-Viger, Gomariz, Gueniche, Mwamikazi, & Thomas, 2013; Huynh, Vo, & Snásel, 2017; Le, Tran, & Vo, 2015; Pei et al., 2004; Tzvetkov, Yan, & Han, 2005; Yang, Wang, & Kitsuregawa, 2007). These methods have been applied to many real-world applications, such as classification (Fradkin & Mörchen 2015), mining repetitive patterns (Tong et al., 2009), protein localization prediction (Baralis, Chiusano, & Dutto, 2008), activity prediction (Ceci, Lanotte, Fumarola, Cavallo, & Malerba, 2014), and medical prescription prediction (Wright, Wright, McCoy, & Sittig, 2015). Mining FSPs from large databases is very important in many areas with broad applications. Huge amounts of data, such as stock market data, banking data, customer data, and hospital data, are stored in sequential databases. Therefore, the mining of FSPs plays an essential role in many data mining tasks such as mining associations, correlations, causality, and trending.

According to the definition given by Agrawal and Srikant (1995), a pattern becomes frequent when its support is not less than a *minsup* threshold. The challenges in FSP mining are: (1) the large number of generated patterns from large databases, (2) multiple database scans, and (3) a combinatorically explosive number of candidates. In many applications, including stock sequence analysis and trend prediction, databases often contain a large number of sequential patterns, many of which are very long. Therefore, efficient and scalable methods for mining FSPs are required.

* Corresponding author at: Ton Duc Thang University, Ho Chi Minh City, Viet Nam.
E-mail addresses: kvttung@gmail.com (T. Kieu), bayvodinh@gmail.com (B. Vo), lecungtuong@tdt.edu.vn (T. Le), zhdeng@cis.pku.edu.cn (Z.-H. Deng), lhbac@fit.hcmus.edu.vn (B. Le).

http://dx.doi.org/10.1016/j.eswa.2017.05.021
0957-4174/© 2017 Elsevier Ltd. All rights reserved.

In some data mining tasks, all patterns in the database are found. However, in some systems, such as recommender systems, expert systems, business intelligence systems, and social networking, end users need to find itemsets that occur with a sequential pattern. For example, in online cart systems, a number of products related to the current products are shown (Berry & Linoff, 1997). When a user purchases products, the system should instantly recommend other products (top-*k* products, for example) that have been bought together with those products. For example, given a user transaction ⟨(*coffee,sugar*)(*tea*)⟩, the system can recommend some products that occur frequently with ⟨(*coffee,sugar*)(*tea*)⟩. The threshold *k*, which depends on store location and the market situation, determines the number of results.

Many researchers have investigated FSP mining (Ayres et al., 2002; Xue et al., 2015; Yang et al., 2007; Zaki, 2001; Zaki & Gouda, 2003), frequent closed inter-sequence pattern mining (Han, Pei, & Yin, 2000), top-*k* frequent pattern mining (Amphawan & Lenca, 2015; Chuang, Huang, & Chen, 2008; Huynh, Le, Vo, & Le, 2015), and top-*k* sequential (closed) pattern mining (Fournier-Viger et al., 2013; Tzvetkov et al., 2005). However, it is very difficult to modify the proposed methods for mining top-*k* co-occurrence items with a sequential pattern because they are based on thresholds (*minimum support* and *minimum confidence*). Mining top-*k* co-occurrence items with a sequential pattern can be treated as mining top-*k* sequential rules to explore *k* rules $X \rightarrow Y$. However, with top-*k* sequential rule mining algorithms, end users cannot choose the constraints on antecedent *X*. Algorithms for top-*k* co-occurrence pattern mining are based on two parameters, namely *k* and pattern *P*. The results are *k* patterns that occur most frequently after pattern *P* in all sequences.

The naive approach for finding top-*k* co-occurrence items with a sequential pattern is to scan all sequences, even those that do not occur with *P*. This approach uses a lot of resources to scan the database, especially for large databases. In addition, it encounters the large search space problem, especially when databases are very large and a long query pattern *P* is used to find top-*k* co-occurrence items. One solution is to apply a pruning strategy to minimize the search space and to efficiently find top-*k* co-occurrence items with a sequential pattern. To mine top-*k* co-occurrence items with a sequential pattern, we propose three algorithms, namely Naive Approach Mining (NAM), Vertical Approach Mining (VAM), and Vertical with Index Approach Mining (VIAM) algorithms, and two data structures. NAM discovers top-*k* co-occurrence items by directly scanning the original database. VAM finds top-*k* co-occurrence items by employing Sid-sets, a type of vertical database, to prune the search space. VIAM, based on VAM, finds top-*k* co-occurrence items by employing a data structure called Sid-sets with an index, a type of enhanced vertical database, to prune the search space. The Sid-sets with an index structure provides a good platform for designing novel strategies for pruning the search space to improve efficiency.

The rest of this paper is organized as follows. Section 1 introduces the problem of mining top-*k* co-occurrence items with a sequential pattern. Section 2 presents some works related to top-*k* mining and FSP mining. Section 3 formalizes the problem in a mathematical representation. In Section 4, NAM, VAM, and VIAM are proposed. The complexity of the three algorithms is also discussed. Section 5 presents the results of extensive experiments conducted with a number of popular databases in data mining to evaluate the performance of the three algorithms. The experimental results show that VIAM is best in terms of execution time, especially when the database is large and dense. Section 6 presents the conclusions, the practical applications of top-*k* co-occurrence items with a sequential pattern, and suggestions for the further development of mining top-*k* co-occurrence items with a sequential pattern.

2. Related work

The problem of mining FSPs is to find the complete set of sequential patterns that satisfy *minsup* in a transaction database (Agrawal & Srikant 1995). Many methods have been proposed for mining FSPs (Ayres et al., 2002; Fournier-Viger et al., 2013; Le et al., 2015; Pei et al., 2004; Tzvetkov et al., 2005;Yang et al., 2007). A property named the downward closure, derived from frequent itemset (FI) mining to prune infrequent patterns, is applied in these approaches. This property states that if a pattern is infrequent, all of its supersets are also infrequent. Besides, the problem of mining FIs was introduced and there are many approaches to solve this problem. A well-known algorithm for mining FIs is the Apriori algorithm (Agrawal & Srikant 1994). Apriori employs a candidate generate-and-test strategy to discover FIs. However, this algorithm requires scanning the database many times (Han et al., 2000). Another important approach for avoiding multiple database scans is using a vertical database format. Zaki et al. employed Tid-sets and diffsets (Zaki, 2001; Zaki & Gouda 2003) to represent the database for FI mining tasks. The supports of itemsets can be calculated from the intersection of itemset Tid-sets or diffsets. Therefore, these approaches scan the database only once. Vo, Le, Coenen, and Hong (2016) proposed an efficient approach for mining FIs using N-list structure and subsume concept. Similarly, for mining FSPs, to avoid scanning the whole database repeatedly, a number of vertical mining algorithms have been proposed. Yang et al. (2007) employed last position inductions (*S – step*), which use the last position of an item to optimize FSPs with the LAPIN algorithm. Recently, some algorithms, such as CM-SPADE and CM-SPAM (Fournier-Viger, Gomariz, Campos, & Thomas, 2014), have used a vertical data format with some additional information to boost performance.

Another problem that hampers the use of FSP mining is that it is difficult for users to choose a proper *minsup* value. To limit the number of FSPs, a good solution is to mine the top-*k* FSPs, as done by TKS algorithm (Fournier-Viger et al., 2013), or frequent closed sequential patterns, as done by TSP algorithm (Tzvetkov et al., 2005) instead of mining all FSPs. Setting *k* is more intuitive than setting *minsup* since *k* represents the number of patterns that the user wants to find whereas choosing *minsup* depends solely on a database's characteristics, which are often unknown to users. Because of the easiness of end-user, nowadays, many top-*k* FSPs mining algorithms have been proposed. For example, focusing on the timestamps of items, mining top-*k* periodic-frequent patterns (Amphawan, Lenca, & Surarerks, 2009) and mining top-*k* regular-frequent itemsets (Amphawan, Lenca, & Surarerks, 2011) have been proposed. Both approaches require two parameters, the number *k* of expected outputs and a maximum periodicity/regularity threshold. In addition, co-occurrence information was defined as an element of the CMAP data structure in the study of Fournier-Viger et al. (2014) and was applied for mining sequential patterns through the extension step of CMAP. Co-occurrence information is used in the present study.

Although FSPs mining algorithms are distinct in that they employ different data structures and search strategies to discover top-*k* frequent patterns, they follow the same general process for discovering top-*k* patterns. The main idea is to maintain a list of patterns *L* and set a *minsup*. *L* contains the top-*k* frequent patterns that have been found so far. By setting the value of *minsup* to the smallest support value of all patterns in *L*, the search space can be greatly pruned when searching for more patterns.

Deng (2015) proposed the idea of mining top-*k* co-occurrence items from transaction databases and developed a number of algorithms. However, it is very difficult to adapt these algorithms for mining top-*k* co-occurrence items with a sequential pattern due

Table 1
Example database (DB_E).

Sid	Customer sequence
1	$\langle(de)(c)(b)(a)(f)\rangle$
2	$\langle(cf)(ab)\rangle$
3	$\langle(bde)(bc)(c)(d)(ac)\rangle$
4	$\langle(a)(bc)(f)(a)\rangle$
5	$\langle(f)(e)(bd)(ac)\rangle$

to the difference between the ordinal characteristics of sequential databases and those of transaction databases.

3. Definitions and problem statement

In this section, we formally define the problem of mining top-k co-occurrence items with a sequential pattern. Let $I=\{i_1, i_2, \ldots, i_t\}$ be the set of universal items. Let s be an itemset (i.e., a set of items). The itemset s is said to be of size l if it contains l ($1\le l\le t$) items, where t is the *size* of the set of universal items. Let $DB=\{S_1, S_2, \ldots, S_n\}$ be a sequential database, where each S_k ($1\le k\le n$) is a sequence e.g. a set of items such that $s\subseteq S_k$.

Example 1. Consider a simplified database for an online shop. The database stores some sequences, as shown on Table 1, where *Sid* and *Customer sequence* are attributes with domains {1, 2, 3, 4, 5} and {*a,b,c,d,e,f*}, respectively. Each sequence S in database DB has a unique identifier (*Sid*) and is a ordered subset of {*a,b,c,d,e,f*}.

For simplicity, we represent sequence S with $Sid=i$ as S_i. For example, S_2 is the second sequence in Table 1. The top-2 items co-occurring with pattern $P=\langle(de)(b)\rangle$ are (c) and (a), because they occur most frequently after P in DB_E. Similarly, the top-1 item co-occurring with P is (c) because it occurs three times after P in sequence S_3 whereas (a) occurs only two times in S_1 and S_3. The top-1 item co-occurring with pattern $P=\langle(c)\rangle$ is item (a) because (a), (b), (d), and (f) occur after $\langle(c)\rangle$, but (a) occurs five times in all sequences. Note that the pattern has an order; in S_5, (a) and (c) occur after (e)(bd). Nevertheless, neither (a) nor (c) in S_5 appear in the results if we use pattern $P=\langle(de)(b)\rangle$. In addition, $\langle(de)(b)\rangle$ is different from $\langle(e)(bd)\rangle$.

Definition 1. Let $P=\langle s_1, s_2, \ldots, s_m\rangle$ be a pattern. A sequence $S=\langle s'_1, s'_2, \ldots, s'_n\rangle$ ($n\ge m$) is called a supersequence of P or P is called a subsequence of S, denoted as $P\sqsubseteq S$, if and only if there exist integers $1\le j_1\le j_2\le\ldots\le j_m\le n$ such that $s_1\subseteq s'_{j1}, s_2\subseteq s'_{j2},\ldots, s_m\subseteq s'_{jm}$.

Definition 2. The *support count* of P, denoted as $SC(P)$, is the number of sequences $S\in DB$ that are supersequences of P.

Definition 3. Given a pattern $P=\langle s_1, s_2, \ldots, s_m\rangle$, a sequence $P^*=\langle s'_1, s'_2, \ldots, s'_n\rangle$, ($n\le m$), is called the *prefix* of P if and only if (1) $s'_i=s_i$ ($i\le m-1$), (2) $s'_i\subseteq s_i$, and (3) all the frequent items in $(s_i-s'_i)$ are alphabetically after those in s'_i (Pei et al., 2004).

Definition 4. Given a pattern $P=\langle s_1, s_2, \ldots, s_m\rangle$, a sequence $P^*=\langle s'_1, s'_2, \ldots, s_{n-1}, s'_n\rangle$, ($n\le m$), is the *prefix* of P. Sequence $\gamma=\langle s''_n, s''_{n+1}, \ldots, s_m\rangle$ is called the *suffix* of P with regards to prefix P^*, denoted as $\gamma=P/P^*$. We also denote $P=P^*\cdot\gamma$ (Pei et al., 2004).

Definition 5. Given a pattern P, an item i ($i\in I\wedge i\notin P$) is a co-occurrence item of P if at least one sequence S in DB is a *supersequence* of $P\wedge\{i\}\in S$, where P is the *prefix* of i in sequence S. The set of all co-occurrence items with P is denoted as P_{coi}.

Definition 6. Given the sequence S and itemset s, $s\sqsubseteq S$, the set of indices of an itemset s is the set of positions of s in S, denoted as $INDEX(s,S)=\{positions\ of\ s\ in\ S\}$.

For example, for $S=\langle(de)(c)(b)(de)(a)(f)\rangle$, $INDEX((de),S)=\{1,4\}$.

Definition 7. Given an item i and sequence S, the set of indices of item i are denoted as:

$$ITEM_INDEX(i,S)=\{INDEX(s,S)|i\in s\wedge s\sqsubseteq S\}$$

For example, for $S=\langle(de)(c)(b)(de)(a)(f)\rangle$, $ITEM_INDEX((d),S)=\{1,4\}$, $ITEM_INDEX((e),S)=\{1,4\}$, $ITEM_INDEX((c),S)=2$, $ITEM_INDEX((b),S)=3$, $ITEM_INDEX((a),S)=4$, and $ITEM_INDEX((f),S)=5$.

Definition 8. The minimum index of an itemset s in sequence S, denoted as $MI(s,S)$, is defined as $MIN(INDEX(s,S))$.

For example, given $S=\langle(de)(c)(de)(b)(c)(a)(f)\rangle$, we have $MI((de),S)=1$ and $MI((c),S)=2$.

Definition 9. Given a pattern $P=\langle s_1, s_2, \ldots, s_m\rangle$ and $P\sqsubseteq S$, the minimum index of P in S, denoted as $MI(P,S)$, is defined as the smallest index of the 1-sequence *suffix* of $P^*=\langle s_1, s_2, \ldots, s_{m-1}\rangle$ in sequence S, that is $MIN(INDEX(s_m,S))$.

For example, given $S=\langle(bde)(bc)(c)(d)(ac)\rangle$ and $P=\langle(de)(b)\rangle$, we have $MI(P,S)=2$.

Definition 10. Given a pattern $P=\langle s_1, s_2, \ldots, s_m\rangle$ and sequence S with $P\sqsubseteq S$, the effective length of P in S, denoted as $LEN(P,S)$, is defined as: $LEN(P,S)=MI(P,S)-MI(s_1,S)$.

Property 1. Given pattern P and sequence S, then $\forall P\sqsubseteq S\rightarrow LEN(P,S)\ge 0$.

Proof. $|P|$, the length of pattern P, is defined as the number of itemsets s contained in P. There are two cases:

1 If $|P|=1$, we have $MI(P,S)=MI(s_1,S)$; therefore, $LEN(P,S)=0$.
2 If $|P|>1$, the first position of s_m that appears after s_1 in S (i.e., $MI(P,S)$) must be greater than the first position of s_1 in S (i.e., $MI(s_1,S)$). Therefore, $MI(P,S)>MI(s_1,S)$ or $LEN(P,S)>0$.

Lemma 1. Given a sequence S and a pattern P with $P\sqsubseteq S$, if the item i is a co-occurrence item with pattern P, we always have $\exists ITEM_INDEX(i_\alpha,S)\in ITEM_INDEX(i,S)\rightarrow ITEM_INDEX(i_\alpha,S)\ge MI(P,S)$.

Proof. As defined in Definition 3, an item i ($i\in I\wedge i\notin P$) is a co-occurrence item of P if and only if one or more sequences in DB contain $P\cup\{i\}$, where P is the *prefix* of i in sequence S. With sequence S, pattern P, $P\sqsubseteq S$, and **Definition 6**, $MI(P,S)$ is the smallest index of the 1-sequence *suffix* of $P^*=\langle s_1, s_2, s_{m-1}\rangle$ in sequence S. From Definition 3 and Definition 6, if item i is a co-occurrence item with pattern P, $\exists ITEM_INDEX(i_\alpha,S)\in ITEM_INDEX(i,S)\rightarrow ITEM_INDEX(i_\alpha,S)\ge MI(P,S)$.

Based on Lemma 1, to mine co-occurrence items with a sequential pattern, we have to consider items $i\leftarrow ITEM_INDEX(i_\alpha,S)\in ITEM_INDEX(i,S)\ge MI(P,S)$. Items $j\leftarrow\forall ITEM_INDEX(j_\alpha,S)\in ITEM_INDEX(j,S)<MI(P,S)$ do not need to be checked as co-occurrence items of P.

Definition 11. Given pattern P and i ($\in P_{coi}$), the co-occurrence count of P and i, denoted as $COUNT(P,i)$, is defined as the *support count* of $P\cup i(\in P_{coi})$.

Definition 12. Given pattern P, an item $i(\in P_{coi})$ is called a top-k co-occurrence item of P if $i\in R_{Tk}=\{i_1, i_2, \ldots, i_g\}(n\le k)\wedge\nexists i'\in I/R_{Tk}=\{i'_{g+1}, i'_{g+2}, \ldots, i'_h\}\rightarrow COUNT(P,i')>COUNT(P,i)$.

With Definition 12, we define the problem of mining top-k co-occurrence items with a sequential pattern as follows. Given a sequence database DB, pattern P, and threshold k, the problem of mining the complete set of top-k co-occurrence items with P is to

Algorithm 1: NAM algorithm.

Input: Sequential database *DB*, pattern *P*, and threshold *k*.
Output: R_{Tk}, the set of all top-*k* co-occurrence items of *P*.
1: **foreach** $S \in DB$ **do**
2: **if** $P \sqsubseteq S$ **then**
3: **foreach** $i \in S - P$ and $ITEM_INDEX(i,S) \geq MI(P,S)$ **do**
4: **if** *i* is first visited **then**
5: $COUNT(i) \leftarrow 1$;
6: **else**
7: $COUNT(i) \leftarrow COUNT(i)+1$;
8: **end if**
9: **end for**
10: **end if**
11: **end for**
12: $R_{Tk} \leftarrow \{x | COUNT(x)$ is one of top-*k* (largest) elements of $\{COUNT(i)\}\}$;
13: **return** R_{Tk};

find the set R_{Tk} containing *k* items in *DB*. For each item $i_a \in Rtk | i_a \notin P \wedge P \cup \{i_a\} \subseteq S \wedge (\exists i_a \rightarrow ITEM_INDEX(i_a,S) \geq MI(P,S))$.

Problem statement. Let us sort all co-occurrence items of *P* by descending *COUNT* value and let $COUNT_k$ be the *support count* of the kth item in the sorted list. The top-*k* co-occurrence items are the set of the first *k* items having the highest *support count* (their *support count* values are greater than or equal to $COUNT_k$).

Therefore, the top-*k* co-occurrence items with sequential pattern mining problem is to discover *k* items with the highest *support count* from *DB* with two user-given parameters: the number *k* of expected outputs and the pattern *P*.

Assume that we want to know the most frequent items of $\langle(a)(c)\rangle$ in Table 1. According to Definition 8, we know that $COUNT(\langle(a)(c)\rangle,b)=1$ because only S_2 contains $\langle(a)(c)\rangle$. Similarly, we have $COUNT(\langle(a)(c)\rangle,d)=2$, $COUNT(\langle(a)(c)\rangle,e)=1$, and $COUNT(\langle(a)(c)\rangle,f)=3$. If threshold *k* is set to 1, the top-1 co-occurrence item with $\langle(a)(c)\rangle$ is (*f*). If *k* is set to 2, the top-2 co-occurrence items with $\langle(a)(c)\rangle$ are (*f*) and (*d*).

4. Mining top-*k* co-occurrence items with sequential pattern

In this section, we describe the three proposed algorithms, namely NAM, VAM, and VIAM, for finding top-*k* co-occurrence items with a given sequential pattern. NAM is the baseline approach, VAM uses a vertical database structure, and VIAM is an enhanced extension of VAM that uses a vertical database structure with an index.

4.1. NAM algorithm

Given a pattern *P* and a database *DB*, NAM scans every sequence in *DB*. If a sequence contains *P*, NAM increases the co-occurrence count of each item *i* that appears after *P*. By visiting all sequences, NAM obtains co-occurrence counts of all co-occurrence items of *P*. By sorting all co-occurrence items in descending order of co-occurrence count, NAM can find all top-*k* co-occurrence items of *P*. Algorithm 1 shows the details of NAM.

First, the NAM algorithm scans all sequences (line 1) and finds all sequences that contain pattern *P* (line 2). Then, the algorithm finds all co-occurrence items with pattern *P* (line 3) and calculates the co-occurrence support count of item *i* (lines 4–7). Finally, the algorithm returns *k* items (lines 12–13).

Example 2. Consider the database in Table 1, pattern $P=\langle(de)(b)\rangle$, and k=1. NAM first scans the whole database. The result is sequences, which are supersequences of *P*, e.g. S_1 and S_3. Next, with sequences S_1 and S_3, NAM gets item $i \notin P \wedge ITEM_INDEX(i,S) \geq MI(P,S)$. In sequence S_1, items (*a*) and (*f*) occur one time after *P*. In sequence S_3, item (*c*) occurs three times and item (*a*) occurs one time after *P*. Finally, item (*c*) occurs three times ($COUNT(c)=3$), item (*a*) occurs two times ($COUNT(a)=2$), and item (*f*) occurs one time ($COUNT(f)=1$). With k=1, top-*k* co-occurrence item with *P* is (*c*).

Algorithm 2: VAM algorithm.

Input: Sequential database *DB*, pattern *P*, and threshold *k*.
Output: R_{Tk}, the set of all top-*k* co-occurrence items of *P*.
1: **foreach** $S \in DB$ **do**
2: **foreach** $i \in S$ **do**
3: **if** *i* is first visited **then**
4: $Sid_list(i) \leftarrow \{S.Sid\}$;
5: **else**
6: $Sid_list(i) \leftarrow Sid_list(i) \cup \{S.Sid\}$;
7: **end if**
8: **end for**
9: **end for**
10: $Sid_list(P) \leftarrow NULL$;
11: **foreach** $i \in P$ **do**
12: **if** *i* is first visited **then**
13: $Sid_list(P) \leftarrow Sid_list(i)$;
14: **else**
15: $Sid_list(P) \leftarrow VAM_INTERSECT(Sid_list(P),Sid_list(i))$;
16: **end if**
17: **end for**
18: $DBP \leftarrow \{S | S \in DB \cap S.Sid \in Sid_list(P)$;
19: **foreach** $S \in DBP$ **do**
20: **if** $P \sqsubseteq S$ **then**
21: **foreach** $i \in S-P \wedge ITEM_INDEX(i,S) \geq MI(P,S)$ **do**
22: **if** *i* is first visited **then**
23: $COUNT(i) \leftarrow 1$;
24: **else**
25: $COUNT(i) \leftarrow COUNT(i)+1$;
26: **end if**
27: **end for**
28: **end if**
29: **end for**
30: $R_{Tk} \leftarrow \{x | COUNT(x)$ is one of top-*k* (largest) elements of $\{COUNT(i)\}\}$;
31: **return** R_{Tk};

4.2. VAM algorithm

NAM is inefficient because it scans all sequences in *DB*, even those that are not supersequences of pattern *P*. This problem is very significant for a large database. Based on a vertical database structure (Zaki, 2001), VAM avoids visiting undesirable sequences. First, VAM converts the original database into a vertical database, where *Sid* is the items in the original database and *Items* is the Sid in the original database. VAM uses Sid-sets, a data structure that originates from an inverted list. The Sid-set of an item is the set of *Sids* of all sequences that contain that item. To employ Sid-sets, a conversion is necessary. In addition, Sid-sets is an appropriate approach in the real world because in the real world, the number of items is usually smaller than the number of sequences. For example, a supermarket may have 5000 products (items) and 100,000 daily customers (sequences).

VAM algorithm comprises two elements (Algorithms 2 and 3). Algorithm 2 is the main elements of VAM Algorithm while Algorithm 3 is named *VAM_INTERSECT* which is an important function of VAM algorithm. *VAM_INTERSECT* is the intersection operator. Note that the intersection is processed in the order of items in pattern *P*. For example, for $P=\langle(e),(b),(f)\rangle$, the result is $VAM_INTERSECT(VAM_INTERSECT((SID_list(e),SID_list(b)),SID_list(f))=\{1,5\}$. Table 2 shows the results of converting Table 1 to Sid-sets.

Algorithm 2 describes the VAM algorithm.

First, the VAM algorithm scans all sequences and all items to convert the database into a vertical format (lines 1–9). Then, it scans all items *i* contained in *P* and calls the *VAM_INTERSECT* function to create filtered database *DBP* (lines 11–18). *VAM_INTERSECT* is the main idea of this algorithm. This function creates the filtered database, which contains all items of pattern *P*. Finally, VAM uses

Algorithm 3: *VAM_INTERSECT*(*Sid_list*(*P*),*Sid_list*(*i*)) function.

```
foreach m ∈ Sid_list(P) do
  foreach n ∈ Sid_list(i) do
    if m = n then
      Sid_list(Q) ← m;
    end if
  end for
end for
return Sid_list(Q);
```

Table 2
Vertical version of DB_E.

Item	Sid
a	1, 2, 3, 4, 5
b	1, 2, 3, 4, 5
c	1, 2, 3, 4, 5
d	1, 3, 5
e	1, 3, 5
f	1, 2, 4, 5

Table 3
First positions of items.

Sid	Itemsets
1	(*de*)(*c*)(*b*)(*a*)(*f*)
2	(*cf*)(*ab*)
3	(*bde*)(*c)(*)(*)(*a**)
4	(*a*)(*bc*)(*f*)(*)
5	(*f*)(*e*)(*bd*)(*ac*)

DBP to find top-*k* co-occurrence items with pattern *P* (lines 19–31) using a method similar to that used by the NAM algorithm.

Example 3. Lines 1–8 in the VAM algorithm are the preprocessing phase and lines 19–31 are the processing phase. Lines 1–9 in the preprocessing phase convert the database to Sid-sets. Lines 10–18 use Table 2 for the input. Consider the database in Table 1, pattern $P=\langle(de)(b)\rangle$, and k=1. First, *Sid_list*(*P*) is created to store the *Sid* of an item in pattern *P*. Then, *Sid_list*(*P*) stores the Sid of the first item in pattern $P=\langle(de)(b)\rangle$, which is (*d*). Therefore, *Sid* 1, 3, 5 is put in *Sid_list*(*P*). The *Sid* of the next items in pattern $P=\langle(de)(b)\rangle$, which is (*e*), is determined. Therefore, *Sid* 1, 3, 5 is put in *Sid_list*(*P*) through the *VAM_INTERSECT* function. *VAM_INTERSECT* only adds *Sid* if it exists in both *Sid_list*(*P*) and processing Sid-set. In this case, no *Sid* is removed. Similarly, item (*b*) is processed. *Sid_list*(*b*), which contains *Sid* 1, 2, 3, 4, 5, is then intersected with *Sid_list*(*P*), which contains *Sid* 1, 3, 5. After this operation, *Sid_list*(*P*) will contain *Sid* 1, 3, 5. *Sid_list*(*P*) is used to filter *DB* to make *DBP*. *DBP* is the pruned database that stores the sequences that contain the items in pattern *P*. These sequences are S_1, S_3, and S_5. Finally, the VAM algorithm scans *DBP* and processes *DBP* similarly to NAM algorithm. That is, VAM algorithm makes use of NAM algorithm to scan sequences S_1, S_3, and S_5 (Table 3).

4.3. VIAM algorithm

One problem of VAM is that it cannot determine exactly whether *S* is a supersequence of *P* because it only checks whether sequence *S* contains the permutations of *P*. In other words, it only checks whether sequence *S* contains item *i* ($i \in P$). For example, for $P=\langle(a)(e)\rangle$, the algorithm will check whether *S* contains $\langle(a)(e)\rangle$ or $\langle(e)(a)\rangle$. Although *S* contains $\langle(e)(a)\rangle$, *S* will be added to *DBP*. This case leads to redundance in *DBP*. Another problem, as can be seen from Lemma 1, is that when VAM scans a sequence that contains *P*, it must scan the whole sequence from start to end although item *i* co-occurrence with pattern *P* has $\exists ITEM_INDEX(i_\alpha,S) \in ITEM_INDEX(i,S) \geq MI(P,S)$. This produces redundant information, leading to unnecessary scans, especially when *LEN*(*P*,*S*) or *MI*(*P*,*S*) is large.

Example 4. Consider sequence $S=\langle(a)(a)(a)(a)(a)\ldots(a)(e)(b)(e)(d)(b)(f)\rangle$, which contains itemset (*a*) 1000 times. Therefore, *MI*(*P*,*S*) =1000 and $P=\langle(e)(b)\rangle$. Consider sequence $S=\langle(e)(a)(a)(a)\ldots(a)(d)(b)(f)\rangle$, which contains itemset (*a*) 1000 times. Therefore, *LEN*(*P*,*S*) =1000. With NAM or VAM, we must find *MI*(*P*,*S*) to mine top-*k* co-occurrence items with a sequential pattern. Because of the requirement of *MI*(*P*,*S*), we must scan the whole sequence with 1000 meaningless occurrences of item (*a*). Based on *Last Position of Item* which is introduced in LAPIN (Yang et al., 2007), we propose a data structure called Sid-sets with an index. This data structure is similar to Sid-sets, introduced in Section 4.2, Moreover, Sid-sets with an index stores the index of each item in sequence to make use of *First Position f Item* idea. More precisely, based on this data structure, we only store the first occurrence of each *Item* in each *Sid*

Sid-sets with an index also stores the *ITEM_INDEX*(*i*,*S*) of an item in the sequence. This data structure improves efficiency because with the condition $\exists ITEM_INDEX(i_\alpha,S) \in ITEM_INDEX(i,S) > MI(P,S)$, we can jump directly to the index of i_α without scanning from start to end. VIAM uses Sid-sets with an index.

Table 4 shows the results after converting Table 1 to the Sid-sets with an index structure, where {*x*,*y*} is the set, *x* is *Sid*, and *y* is *index*. The VIAM algorithm is shown in Algorithm 4.

First, VIAM scans all sequences and all items to convert the database into a vertical format (lines 1–9). In this phase, VIAM takes into account the indices of all items, so the vertical database format contains the sequences and indices of items. Then, VIAM scans all items *i* contained in *P* and calls the *MIN* and *VIAM_INTERSECT* functions to create filtered database *DBP* (lines 11–18). *MIN* and *VIAM_INTERSECT* are the main ideas of this algorithm. *MIN* is used for calculating the min indices of all items for each sequence. *VIAM_INTERSECT*, an extension of *VAM_INTERSECT*, is used for creating filtered *DBP*, which contains pattern *P*. Finally, VIAM uses *DBP* to find top-*k* co-occurrence items with pattern *P* (lines 19–29).

Example 5. Lines 1–13 in the VIAM algorithm are the preprocessing phase and lines 14–21 are the processing phase. Lines 7–13 use Table 4. With input similar to that in Example 2, *Sid_list_w_index*(*P*) is created to store the *Sid* and *index* of the items in pattern *P*. Then, *Sid_list_w_index*(*P*) stores the *Sid* and *index* of the first item in pattern $P=\langle(de)(b)\rangle$, which is (*d*). Therefore, {1, 1}, {3, 1}, {3, 4}, {5, 3} are processed. The first object in the example set {1, 1} has *Sid*=1 and the second object has *index*=1. Line 10 shows the MIN function, which gets the item with the smallest index in each *Sid*. Then, {1, 1}, {3, 1}, {5, 3} are put in *Sid_list_w_index*(*P*). Next, the second item in pattern *P* is processed. *Sid_list_w_index*(*e*) is {1, 1}, {3, 1}, {5, 2}. *Sid_list_w_index*(*e*) is intersected with *Sid_list_w_index*(*P*) through the *VIAM_INTERSECT* function. Because (*d*) and (*e*) are in the same itemset, *VIAM_INTERSECT* compares and removes sets that do not have the same *index* or *Sid*. That is, {5, 2} and {5, 3} are removed. Similarly, item (*b*) is processed. *Sid_list_w_index*(*b*) is {1, 3}, {2, 2}, {3, 1}, {3, 2}, {4, 2}, {5, 3}. Because (*b*) and (*de*) are not in the same itemset, *VIAM_INTERSECT* removes sets that satisfy both of the following conditions: (1) *Sid* in *Sid_list_w_index*(*b*) equal to sets that have the same *Sid* in *Sid_list_w_index*(*P*), and (2) the index of the sets in *Sid_list_w_index*(*b*) do not greater than index of sets in *Sid_list_w_index*(*P*). In this case, {3, 1} is removed. *VIAM_INTERSECT* also removes sets that have the *Sid* in *Sid_list_w_index*(*b*) but does not appear in *Sid_list_w_index*(*P*) and vice versa. In this case, {2, 2}, {4, 2}, and {5, 3} are removed. After this phase, *Sid_list_w_index*(*Temp*) is created to store a tempo-

Table 4
Sid-sets with index structure of items.

Item	Sids with index
a	{1, 4}, {2, 2}, {3, 5}, {4, 1}, {4, 4}, {5, 4}
b	{1, 3}, {2, 2}, {3, 1}, {3, 2}, {4, 2}, {5, 3}
c	{1, 2}, {2, 1}, {3, 2}, {3, 3}, {3, 5}, {4, 2}
d	{1, 1}, {3, 1}, {3, 4}, {5, 3}
e	{1, 1}, {3, 1}, {5, 2}
f	{1, 5}, {2, 1}, {4, 3}, {5, 1}

Algorithm 4: VIAM algorithm.

Input: Sequential database *DB*, pattern *P*, and threshold *k*.
Output: R_{Tk}, the set of all top-*k* co-occurrence items of *P*.
1: **foreach** $S \in DB$ **do**
2: **foreach** $i \in S$ **do**
3: **if** *i* is first visited **then**
4: $Sid_list_w_index(i) \leftarrow S.Sid,index(i)$;
5: **else**
6: $Sid_list_w_index(i) \leftarrow Sid_list_w_index(i) \cup S.Sid,index(i)$;
7: **end if**
8: **end for**
9: **end for**
10: $Sid_list_w_index(P) \leftarrow NULL$;
11: **foreach** $i \in P$ **do**
12: **if** *i* is first visited **then**
13: $Sid_list_w_index(P) \leftarrow MIN(Sid_list_w_index(i))$;
14: **else**
15: $Sid_list_w_index(P) \leftarrow VIAM_INTERSECT(Sid_list_w_index(P),Sid_list_w_index(i))$;
16: **end if**
17: **end for**
18: $DBP \leftarrow \{S \mid S \in DB \cap S.Sid \in Sid_list_w_index(P), S.startindex \leftarrow S.index \in Sid_list_w_index(P)\}$;
19: **foreach** $S \in DBP$ **do**
20: **foreach** $i \in S - P$ **do**
21: **if** *i* is first visited **then**
22: $COUNT(i) \leftarrow 1$;
23: **else**
24: $COUNT(i) \leftarrow COUNT(i) + 1$;
25: **end if**
26: **end for**
27: **end for**
28: $R_{Tk} \leftarrow \{x \mid COUNT(x)$ is one of top-*k* (largest) elements of $\{COUNT(i)\}\}$;
29: **return** R_{Tk};

rary set of processed $Sid_list_w_index(b)$. {1, 3} and {3, 2} are put in $Sid_list_w_index(Temp)$. The *MIN* function is called to get the set of each *Sid* that has the smallest *index*. In this case, no set is removed and {1, 3} and {3, 2} are put in $Sid_list_w_index(Q)$. Finally, $Sid_list_w_index(Q)$ is used to filter *DB* to make *DBP*. *DBP* is the pruned database that stores the sequences that contain pattern *P*. These sequences are S_1 and S_3. Finally, the VIAM algorithm scans *DBP*. That is, it scans sequences S_1 and S_3. From lines 14 to 19, the algorithm scans directly from *index*=3 to the end of S_1 and from *index*=2 to the end of S_3 to get all items not contained in *P* to find co-occurrence items with *P*.

5. Experimental results

In this section, the performance of the three algorithms is compared. Experiments were performed on a personal computer with

Algorithm 5: $MIN(Sid_list_w_index(i))$ function.

foreach $m \in Sid_list_w_index(i)$ **do**
group by $n \leftarrow m.Sid$;
foreach *n* **do**
$j \leftarrow min(n.index)$;
$Sid_list_w_index(P) \leftarrow j$;
end for
end for
return $Sid_list(P)$;

Algorithm 6: $VIAM_INTERSECT(Sid_list_w_index(P), Sid_list_w_index(i))$ function.

foreach $m \in Sid_list_w_index(P)$ **do**
foreach $n \in Sid_list_w_index(i)$ **do**
if $(m \wedge n) \in P$ **then**
if $m.Sid = n.Sid \wedge m.index = n.index$ **then**
$Sid_list_w_index(Temp) \leftarrow n$;
end if
else
if $m.Sid = n.Sid \wedge m.index < n.index$ **then**
$Sid_list_w_index(Temp) \leftarrow n$;
$Sid_list_w_index(Q) \leftarrow MIN(Sid_list_w_index(Temp))$;
end if
end if
end for
end for
return $Sid_list_w_index(Q)$;

2 GB of memory and a 3.2-GHz Intel Core i5 (4 cores) processor. All codes were implemented in C# with .NET Framework 4.0 and run under 32-bit Windows 7 SP1.

5.1. Databases

Five real-world databases were used in the experiments, namely BMSWebview, Bible, FIFA, MSNBC, and Leviathan, which are often used in FSP mining studies (Fournier-Viger et al., 2013). To test the performance of the proposed algorithms on large-scale databases, we used the full versions of these databases (i.e., no sequences or items were removed). Table 5 shows the characteristics of the databases used in the experiments. The fourth column in Table 5 gives the average length, which is defined as:

$$average\ length = \frac{1}{N}\sum_{i=1}^{N} L_i$$

where L_i is the length of row *i*.

5.2. Algorithm complexity

In this section, we describe and analyze the experimental results in terms of various metrics, including preprocessing time (for VAM and VIAM), memory usage, processing time, and scalability. The complexity of NAM, VAM, and VIAM is $O(n \times a)$, where *n* is the number of sequences and *a* is the average number of items in each sequence. The algorithms have two nested loops; the first loop scans all sequences and the second loop scans all items in each sequence. VAM prunes the search space. It reduces the value of *n* to n'. VIAM also prunes the search space. It reduces the value

Table 5
Database information.

Database	Sequence count	Item count	Average length	Type of data
BMSWebview	77,512	3340	4.62	Web click-stream
FIFA	20,450	2990	34.74	Web click-stream
Bible	36,369	13,905	21.6	Book
MSNBC	31,790	17	13.33	Web click-stream
Leviathan	5834	9025	33.8	Book

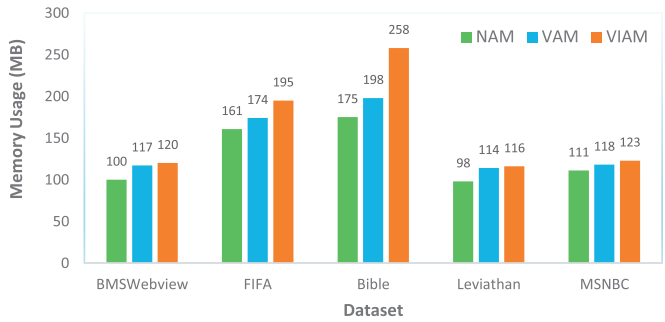

Fig. 1. Memory usage of NAM, VAM, and VIAM for various databases.

of n to n' and reduces the value of a to a'. Although the complexity remains $O(n' \times a')$, pruning significantly reduces processing time and the search space.

Time(*algorithm,input*) is the time required to run an algorithm with a certain input without including the preprocessing time. It is expected that:

$$Time(NAM, input) > Time(VAM, input) > Time(VIAM, input)$$

Memory(*algorithm,input*) is the memory required to run an algorithm with a certain input. It is expected that:

$$Memory(NAM, input) < Memory(VAM, input) < Memory(VIAM, input)$$

Because *Time*(*algorithm,input*) is proportional to the number of sequences, sequence length, and pattern P, the pruning strategy reduces the number of sequences and sequence length at the cost of a preprocessing phase. In addition, a more complex data structure requires more computer resources.

5.3. Memory usage

After a database is loaded into memory, VAM needs to build Sid-sets and VIAM needs to build Sid-sets with an index. This section evaluates the performance of the preprocessing phases. The preprocessing phase begins when a database is loaded into memory and ends when the algorithm is ready to run to obtain the results of a given query pattern. The database, search space, and data structure impact memory usage. The experiment was conducted with several lengths of pattern P. VAM and VIAM have a preprocessing phase to create a vertical database, which consumes memory. VIAM uses more memory because Sid-sets with an index stores both the *Sid* and *index* of an item. Fig. 1 shows the results of the memory evaluation.

5.4. Preprocessing time

The preprocessing time was evaluated for VAM and VIAM. VAM converts the database into Sid-sets and VIAM converts it into Sid-sets with an index. Then, VAM and VIAM prune the Sid-sets. Although the conversion and pruning functions require time, the conversion is not necessary because in the real world, we store data in both original and vertical format. The preprocessing phase is executed once and its results are stored in the vertical database. When a new sequence arises, it will be updated to both the original data and the vertical data at the same time. Figs. 2–6 show the results of the preprocessing time evaluation.

5.5. Processing time

In this section, we evaluate the run-time of the three algorithms. We generated a query pattern with different items. Pattern P includes 1, 5, and 10 itemsets for each case and for each database. For each query, we obtained three run-times and calculated the average run-time. Figs. 7–11 show the results of the processing time evaluation. Because VAM and VIAM are quite similar and both faster than NAM, the vertical axis has a logarithmic scale to demonstrate the efficiency of VIAM.

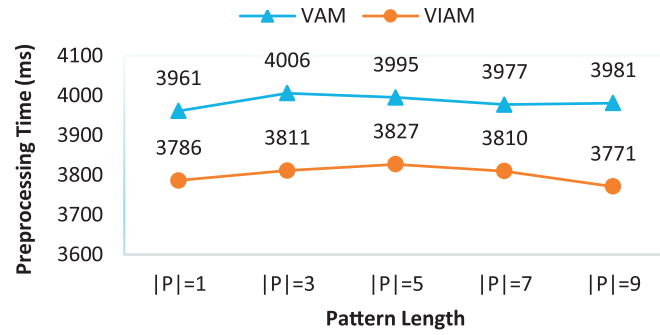

Fig. 2. Preprocessing time of VAM and VIAM for BMSWebview.

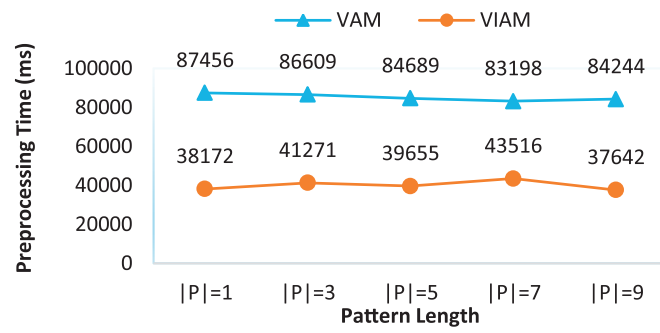

Fig. 3. Preprocessing time of VAM and VIAM for FIFA.

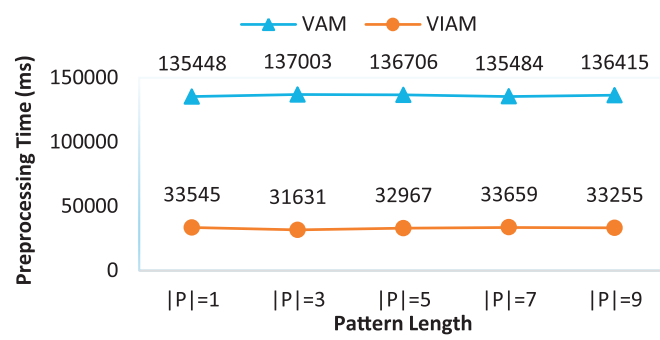

Fig. 4. Preprocessing time of VAM and VIAM for Bible.

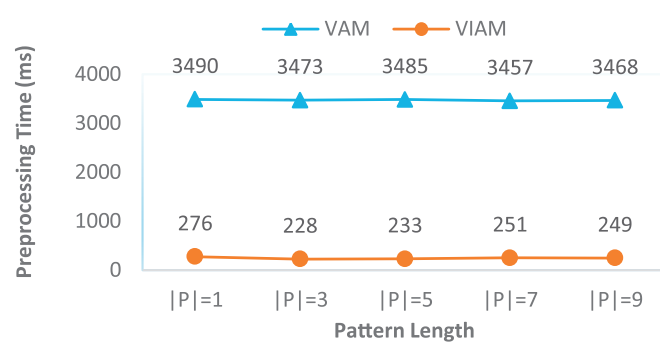

Fig. 5. Preprocessing time of VAM and VIAM for Leviathan.

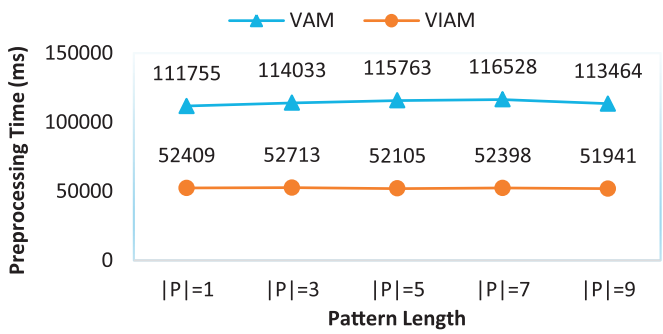

Fig. 6. Preprocessing time of VAM and VIAM for MSNBC.

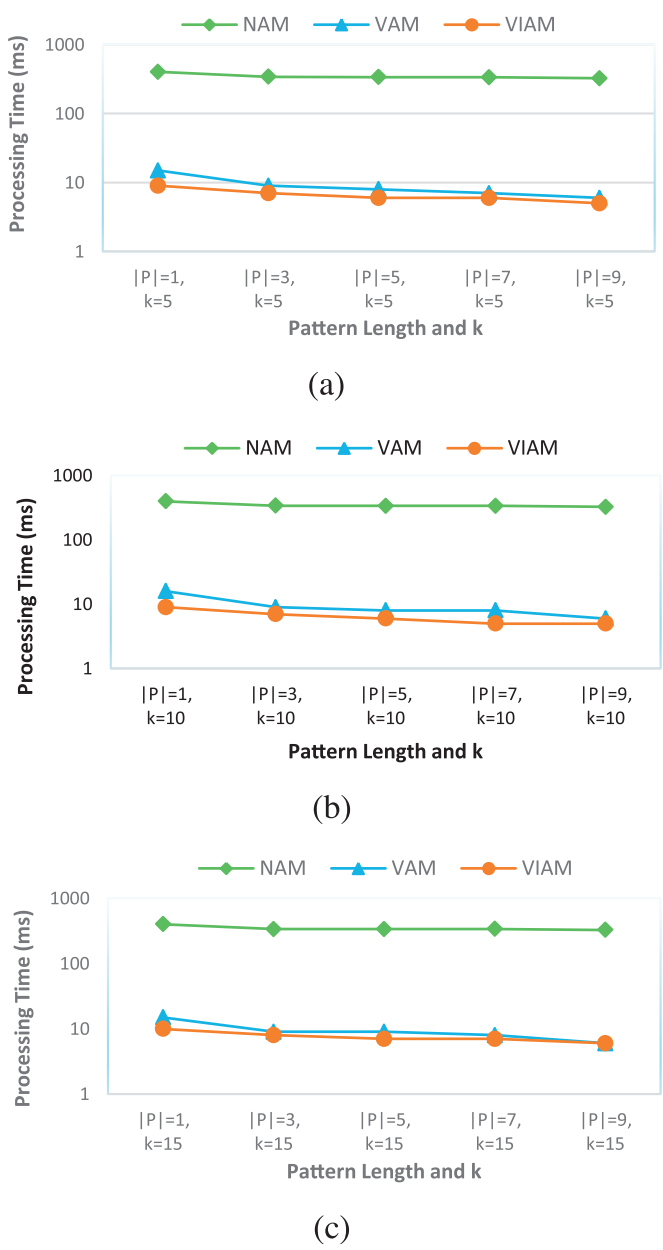

Fig. 7. Processing time of NAM, VAM, and VIAM for BMSWebview with (a) k=5, (b) k=10, and (c) k=15.

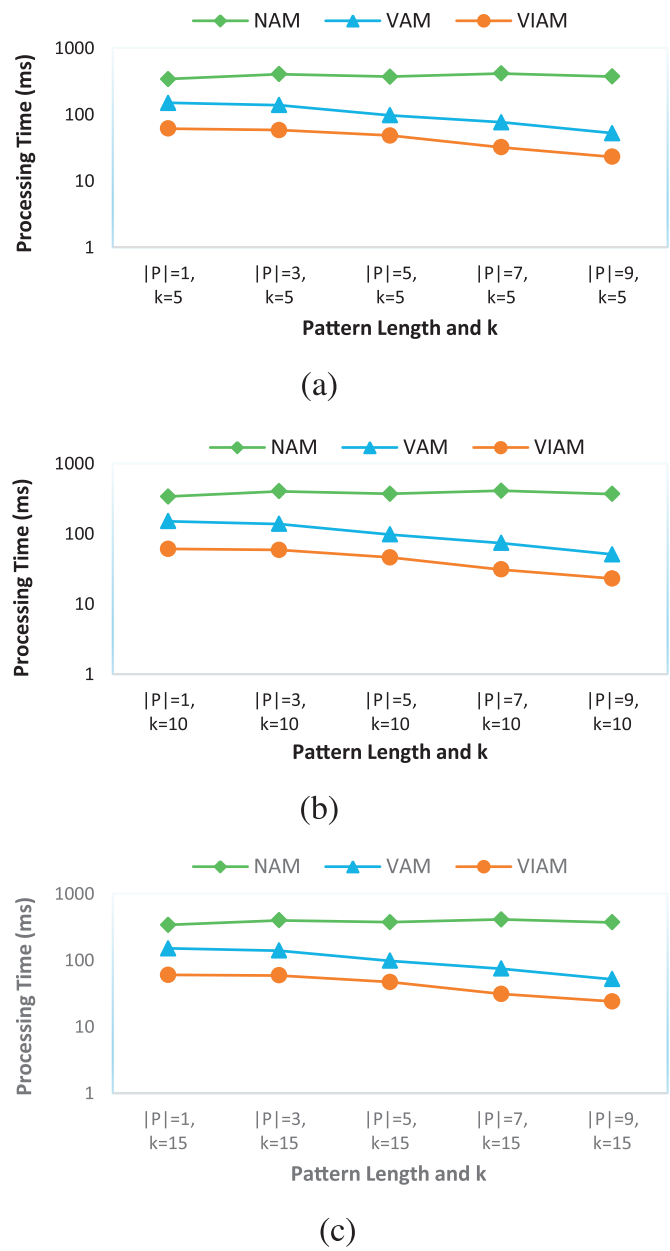

Fig. 8. Processing time of NAM, VAM, and VIAM for FIFA with (a) k=5, (b) k=10, and (c) k=15.

5.6. Discussion

As shown in Section 5.4, threshold k does not affect the preprocessing time because in the preprocessing phase, the algorithms convert and prune the whole database *DB* to create *DBP* without using k. In addition, threshold k does not affect the processing time because the algorithms get all co-occurrence items with pattern P. Therefore, when a user inputs an arbitrary value for k, the algorithms must find all results. Itemset s and item i, which are contained in P, affect the preprocessing and processing time because the preprocessing step applies the intersection operator to each item in P to prune the database, which consumes time. When $|P|$ is larger, the processing time decreases.

When the database has many items and the average length is short, VAM and VIAM reduce processing time. The preprocessing times for VAM and VIAM on FIFA are less than the preprocessing times for these algorithms on Bible. With FIFA, VAM and VIAM

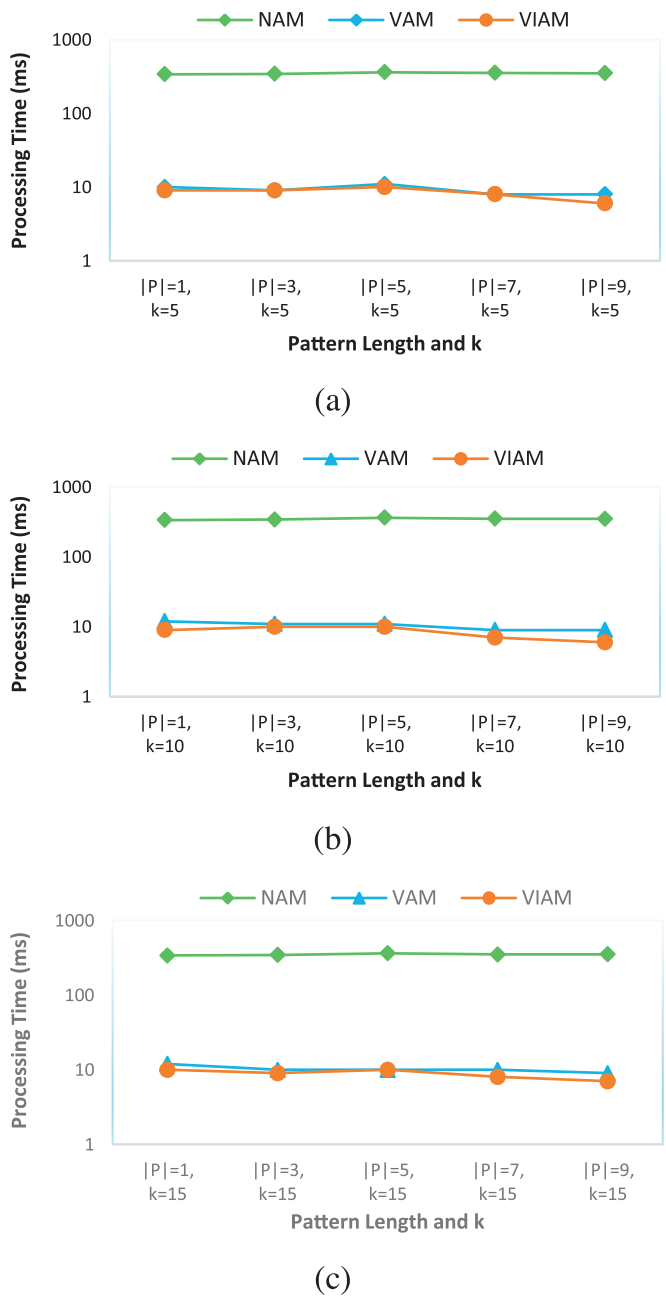

Fig. 9. Processing time of NAM, VAM, and VIAM for Bible with (a) k=5 and (b) k=10, and (c) k=15.

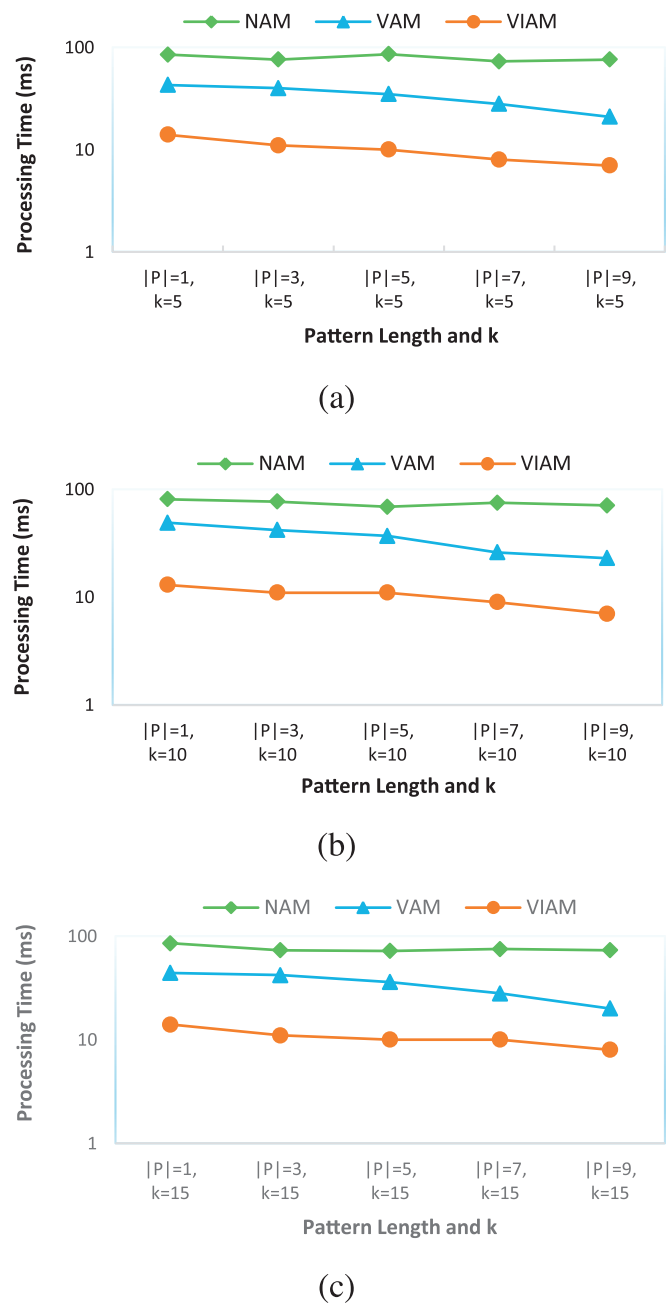

Fig. 10. Processing time of NAM, VAM, and VIAM for Leviathan with (a) k=5, (b) k=10, and (c) k=15.

convert 20,450 sequences to 2990 sequences (number of items). With Bible, VAM and VIAM convert 36,369 sequences to 13,950 sequences (number of items). With a larger number of items and a short average length, the vertical-format-based algorithms prune many sequences that do not contain P. Because of the effect of pruning, the processing times for VAM and VIAM on FIFA are shorter than those on Bible. In addition, when databases have few items and sequences as MSNBC, VAM has a relatively slow processing time. The processing time for VAM with $|P|=1$ and $|P|=3$ are greater than that for NAM on MSNBC. However, with the other $|P|$ values, VAM still promoted the outstanding performance when running for MSNBC.

6. Conclusions and future work

This study developed the NAM, VAM, and VIAM algorithms for mining top-k co-occurrence items with a sequential pattern. The main performance bottleneck of the VAM algorithm is the evaluation of candidates that do not appear in the input database. To address this problem, we proposed a data structure called Sid-sets with an index for storing co-occurrence information. VIAM incorporates Sid-sets with an index to prune the search space and achieve high efficiency. In the experiments, real-world databases were used to evaluate the performance of the three proposed algorithms. The experimental results show that VIAM outperforms the other algorithms substantially in term of execution time and that it has excellent scalability.

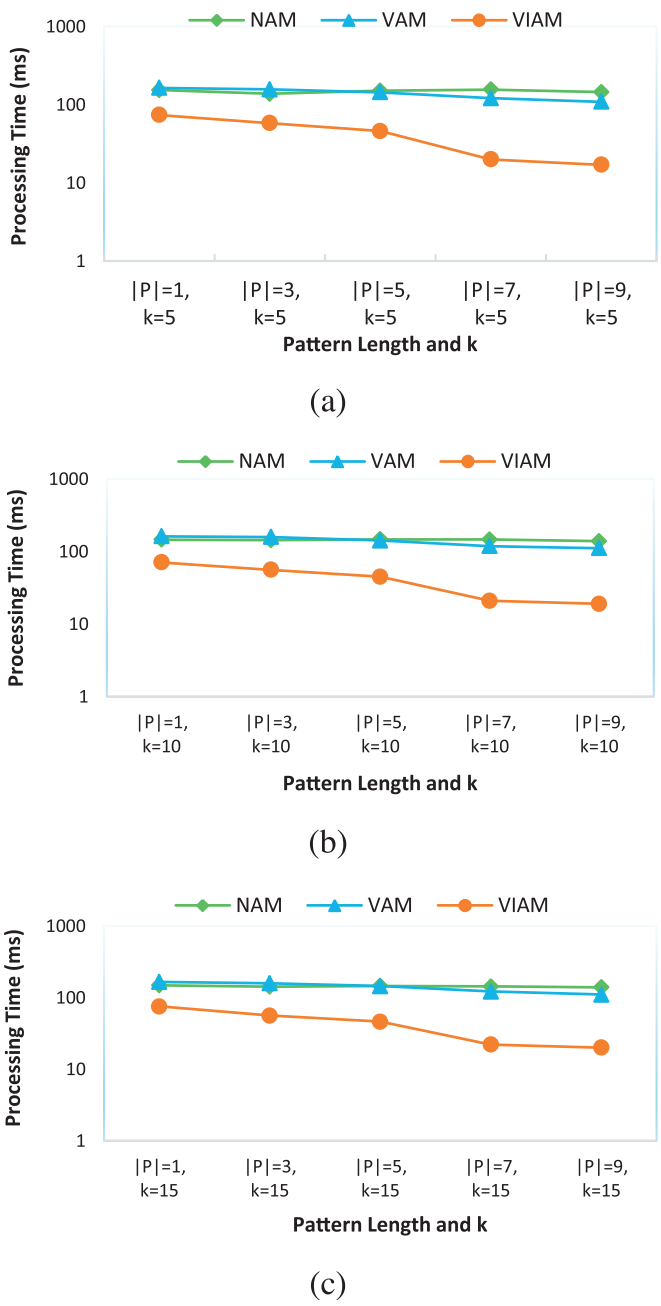

Fig. 11. Processing time of NAM, VAM, and VIAM for MSNBC with (a) k=5 and (b) k=10, and (c) k=15.

In future work, a number of additional optimizations based on data structures such as *Tree*, *Lattice*, and especially *PMAP* (Fournier-Viger et al., 2013) will be developed. We will also study how to adapt and apply these algorithms to real-time databases. Moreover, *bitvectors* will be applied in the processing phase with the intersection operation for mining top-k co-occurrence items with a sequential pattern. Many interesting issues need to be investigated, such as mining co-occurrence items with closed and maximal sequential patterns, mining co-occurrence items with approximate sequential patterns, and mining structured patterns. These studies can be applied to many real-world applications especially in bioinformatics such as DNA sequence mining and protein sequence mining that may admit faults and insertions, deletions, and mutations in DNA sequences. The proposed algorithms can be used for co-occurrence mining in market processing analysis. Using our proposed algorithms to handle industry/engineering sequential processing analysis is also interesting issues for future research.

Acknowledgment

This work was supported by Vietnam's National Foundation for Science and Technology Development (NAFOSTED) under Grant number 102.05-2015.07.

References

Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th international conference on very large data bases, Santiago de Chile, Chile, September 12–15, 1994* (pp. 487–499).

Agrawal, R., & Srikant, R. (1995). Mining sequential patterns. In *Proceedings of the 11th international conference on data engineering, Taipei, Taiwan, March 6–10, 1995* (pp. 3–14).

Amphawan, K., Lenca, P., & Surarerks, A. (2009). Mining top-k periodic-frequent pattern from transactional databases without support threshold. In *Proceedings of the 3rd international conference on advances in information technology, Bangkok, Thailand, December 1–5, 2009* (pp. 18–29).

Amphawan, K., & Lenca, P. (2015). Mining top-k frequent-regular closed patterns. *Expert Systems with Applications, 42*(21), 7882–7894.

Amphawan, K., Lenca, P., & Surarerks, A. (2011). Efficient mining top-k regular-frequent itemset using compressed tidsets. In *Proceedings of the 2011 Pacific-Asia conference on knowledge discovery and data mining workshop, Shenzhen, China, May 24–27, 2011* (pp. 124–135).

Ayres, J., Flannick, J., Gehrke, J., & Yiu, T. (2002). Sequential pattern mining using a bitmap representation. In Proceedings of the 8th ACM SIGKDD international conference on knowledge discovery and data mining, Edmonton, Alberta, Canada, July 23–26, 2002 (pp. 429–435).

Baralis, E., Chiusano, S., & Dutto, R. (2008). Applying sequential rules to protein localization prediction. *Computers & Mathematics with Applications, 55*(5), 867–878.

Berry, M. J., & Linoff, G. (1997). *Data mining techniques: For marketing, sales, and customer support*. New York, NY, USA: John Wiley & Sons, Inc.

Ceci, M., Lanotte, P. F., Fumarola, F., Cavallo, D. P., & Malerba, D. (2014). Completion time and next activity prediction of processes using sequential pattern mining. In *Proceedings of the 17th international conference on discovery science, Bled, Slovenia, October 8–10, 2014* (pp. 49–61).

Chuang, K., Huang, J., & Chen, M. (2008). Mining *top-k* frequent patterns in the presence of the memory constraint. *The VLDB Journal, 17*(5), 1321–1344.

Deng, Z. H. (2015). Mining top-k co-occurrence items, CoRR abs/1512.07806, https://arxiv.org/abs/1512.07806.

Fournier-Viger, P., Gomariz, A., Campos, M., & Thomas, R. (2014). Fast vertical mining of sequential patterns using co-occurrence information. In *Proceedings of the 18th Pacific-Asia conference on advances in knowledge discovery and data mining, Tainan, Taiwan, May 13–16, 2014* (pp. 40–52).

Fournier-Viger, P., Gomariz, A., Gueniche, T., Mwamikazi, E., & Thomas, R. (2013). TKS: Efficient mining of top-k sequential patterns. In *Proceedings of the 9th international conference on advanced data mining and applications, Hangzhou, China, December 14–16, 2013* (pp. 109–120).

Fournier-Viger, P., Lin, J. C. W., Kiran, R. U., Koh, U. S., & Thomas, R. (2017). A survey of sequential pattern mining. *Data Science and Pattern Recognition, 1*(1), 54–77.

Fradkin, D., & Mörchen, F. (2015). Mining sequential patterns for classification. *Knowledge and Information Systems, 45*(3), 731–749.

Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. In Proceedings of the 2000 ACM SIGMOD international conference on management of data, Dallas, Texas, USA, May 16–18, 2000 (pp. 1–12).

Huynh, Q., Le, T., Vo, B., & Le, B. (2015). An efficient and effective algorithm for mining top-rank-k frequent patterns. *Expert Systems with Applications, 42*(1), 156–164.

Huynh, B., Vo, B., & Snásel, V. (2017). An efficient method for mining frequent sequential patterns using multi-core processors. *Applied Intelligence, 46*(3), 703–716.

Le, H. B., Tran, M., & Vo, B. (2015). Mining frequent closed inter-sequence patterns efficiently using dynamic bit vectors. *Applied Intelligence, 43*(1), 74–84.

Pei, J., Han, J., Mortazavi-Asl, B., Wang, J., Pinto, H., Chen, Q., et al. (2004). Mining sequential patterns by pattern-growth: The prefixspan approach. *IEEE Transactions on Knowledge and Data Engineering, 16*(11), 1424–1440.

Tong, Y., Li, Z., Yu, D., Ma, S., Cheng, Z., & Xu, K. (2009). Mining compressed repetitive gapped sequential patterns efficiently. In *Proceedings of the 5th international conference on advanced data mining and applications, Beijing, China, August 17–19, 2009* (pp. 652–660).

Tzvetkov, P., Yan, X., & Han, J. (2005). TSP: Mining top-*k* closed sequential patterns. *Knowledge and Information Systems, 7*(4), 438–457.

Vo, B., Le, T., Coenen, F., & Hong, T. P. (2016). Mining frequent itemsets using the N-list and subsume concepts. *International Journal of Machine Learning & Cybernetics, 7*(2), 253–265.

Wright, A. P., Wright, A., McCoy, A. B., & Sittig, D. F. (2015). The use of sequential pattern mining to predict next prescribed medications. *Journal of Biomedical Informatics, 53*, 73–80.

Xue, Y., Li, T., Liu, Z., Pang, C., Li, M., Liao, Z., et al. (2015). A new approach for the deep order preserving submatrix problem based on sequential pattern mining. *International Journal of Machine Learning and Cybernetics*, 1–17. doi:10.1007/s13042-015-0384-z.

Yang, Z., Wang, Y., & Kitsuregawa, M. (2007). LAPIN: Effective sequential pattern mining algorithms by last position induction for dense databases. In *Proceedings of 12th international conference on database systems for advanced applications: The advances in databases: Concepts, systems and applications, Bangkok, Thailand, April 9–12, 2007* (pp. 1020–1023).

Zaki, M. J. (2001). SPADE: An efficient algorithm for mining frequent sequences. *Machine Learning, 42*(1–2), 31–60.

Zaki, M. J., & Gouda, K. (2003). Fast vertical mining using diffsets. In Proceedings of the ninth ACM SIGKDD international conference on knowledge discovery and data mining, Washington, DC, USA, August 24–27, 2003 (pp. 326–335).